

NAATALFI SAAS — ARCHITECTURE

Guide de Référence Technique Global des Dossiers et Fichiers

Ce document constitue la spécification officielle de l'architecture logicielle de la plateforme **NaatalFi SaaS**. Conçu selon un paradigme modulaire propre (Clean Architecture adaptée à Django et structure d'état découplée pour React), ce squelette assure une isolation stricte des contextes bornés (Bounded Contexts) et une scalabilité optimale pour une infrastructure multi-tenant.

1. RACINE DU PROJET (ROOT)

Fichiers de pilotage de l'environnement, de l'orchestration des conteneurs et de l'intégration continue.

FICHIER	RÔLE TECHNIQUE ET PORTÉE
<code>docker-compose.yml</code>	Orchestration locale des services conteneurisés : application Django, worker Celery, base de données PostgreSQL (miroir Supabase) et instance Redis (Upstash local).
<code>.gitignore</code>	Exclusion des artefacts de build, des secrets d'environnement (<code>.env</code>), des logs, et des médias utilisateur du suivi Git.
<code>README.md</code>	Documentation d'onboarding, commandes d'initialisation, prérequis et instructions de déploiement rapide.

2. SEGMENT BACKEND (DJANGO REST FRAMEWORK)

Le backend implémente une architecture pilotée par le domaine (Domain-Driven Design - DDD léger) où chaque application encapsule son modèle de données, sa logique métier autonome (services) et ses points d'accès (API).

2.1. Noyau de Configuration Global (`backend/config/`)

FICHIER	RESPONSABILITÉ SPÉCIFIQUE
<code>settings.py</code>	Configuration centrale : intégration du middleware d'isolation multi-tenant, routage des bases Supabase, configuration du cache Upstash Redis, et paramétrage du chiffrement des tokens JWT.
<code>urls.py</code>	Passerelle de routage URL racine. Délègue le traitement aux routeurs d'API internes de chaque application via le préfixe réglementaire <code>/api/v1/</code> .
<code>celery.py</code>	Instanciation et configuration du courtier de messages (Message Broker) Celery pour la gestion de la file d'attente asynchrone.
<code>wsgi.py</code> & <code>asgi.py</code>	Points d'entrée des serveurs d'application : WSGI pour le trafic HTTP synchrone classique, ASGI pour les protocoles asynchrones et les futurs WebSockets (notifications temps réel).

2.2. Structure Interne Standardisée d'une Application (App Layout)

Chaque module présent dans `backend/apps/` adopte une subdivision uniforme pour éviter le couplage fort :

- `models/` : Découpage des entités SQL en fichiers distincts au lieu d'un seul fichier monolithique.
- `api/` : Regroupe les `serializers.py` (validation/sérialisation) et les `views.py` (ViewSets DRF).
- `services/` : Contient la logique métier pure (Domain Services), isolée des vues et des modèles pour rester testable de manière unitaire.
- `signals.py` : Écouteurs d'événements du cycle de vie des modèles (ex: instancier un Wallet dès qu'un Vendor est validé).

2.3. Cartographie des Domaines Métier Backend (`apps/`)

`users/`

Gestion fine de l'identité et de la sécurité. Modèle utilisateur personnalisé (AbstractUser), attribution des rôles (Admin, Vendor, Customer), mécanismes RBAC (Role-Based Access Control) et cycle de vie des tokens JWT.

`vendors/`

Gestion du cycle de vie des boutiques (Shops), des configurations multi-vendeurs, du KYC des commerçants, de leur branding spécifique et de l'activation de leurs accès.

categories/ & products/

Gestion du catalogue. Découpage strict des entités entre le produit générique (**product.py**), sa galerie (**product_image.py**) et ses déclinaisons techniques de stocks ou de tailles (**product_variant.py**). **pricing_service.py** calcule dynamiquement les grilles tarifaires et taxes.

orders/ & shipping/

Logique critique de la commande de type "Split-Order". Permet au client de payer un panier global unique, qui est ensuite fragmenté en sous-commandes propres à chaque vendeur. Le module shipping calcule les frais par zone de livraison.

payments/ & wallet/

Moteur financier. Orchestration des flux monétaires avec l'API PayTech, gestion des webhooks de sécurisation et écriture comptable dans les portefeuilles virtuels (solde disponible, gelé pendant la livraison, et historiques de retrait).

reviews/, ads/ & disputes/

Modules de confiance et d'écosystème : modération des avis clients, système de mise en avant publicitaire payante pour les vendeurs (Ads) et gestion des workflows de résolution des litiges (Disputes).

core/ (Middleware, Permissions, Constants...)

Le socle technique réutilisable. Contient **constants/roles.py** (définition stricte immuable des accès), les paginations d'API standardisées, et les middlewares transversaux (sécurité, isolation des données, capture globale des exceptions).

tasks/

Fichiers exécutés par le Worker Celery. Déportation des tâches lourdes : génération de rapports analytiques journaliers, relances automatiques par e-mail, appels aux webhooks bancaires et notifications push.

3. SEGMENT FRONTEND (REACT VIA VITE)

L'application frontend est configurée pour offrir une expérience fluide, réactive et optimisée pour le SEO et les performances mobiles (Mobile-First via TailwindCSS).

DOSSIER / FICHIER	RÔLE AU SEIN DU CYCLE DE VIE FRONTEND
<code>src/pages/</code>	Conteneurs principaux rattachés aux routes. Divisés de façon étanche par rôle (Espace public Marketplace, Vendor-Dashboard sécurisé pour les marchands, Admin-Panel de modération globale).
<code>src/components/</code>	Composants d'UI atomiques réutilisables, isolés de l'état global (ex: cartes de produits, graphiques d'analytics, barres de navigation).
<code>src/layouts/</code>	Squelettes de mise en page structurelle (ex: structure avec barre latérale fixe pour les dashboards, ou mise en page fluide pour le catalogue grand public).
<code>src/routes/ & guards/</code>	Contrôle d'accès applicatif côté client. Les guards interceptent les transitions de routes pour bloquer les utilisateurs non authentifiés ou n'ayant pas le rôle adéquat (ex: interdire l'accès au tableau de bord vendeur à un simple client).
<code>src/store/</code>	Gestion centralisée et immuable de l'état global via Zustand , évitant le prop-drilling pour les données partagées (panier d'achat global, session de l'utilisateur).
<code>src/services/</code>	Couche de communication réseau. api.js configure l'instance Axios globale avec interception des erreurs 401 et injection automatique du header HTTP Authorization: Bearer <token> .